

# LSSGen: Leveraging Latent Space Scaling in Flow and Diffusion for Efficient Text to Image Generation

Jyun-Ze Tang<sup>1</sup> Chih-Fan Hsu<sup>1</sup> Jeng-Lin Li<sup>1</sup> Ming-Ching Chang<sup>2</sup> Wei-Chao Chen<sup>1</sup>

<sup>1</sup>Inventec Corporation, Taipei, Taiwan

<sup>2</sup>University at Albany, State University of New York, NY, USA

<sup>1</sup>{tang.nickct, hsu.chih-fan, li.johncl, chen.wei-chao}@inventec.com, <sup>2</sup>mchang2@albany.edu

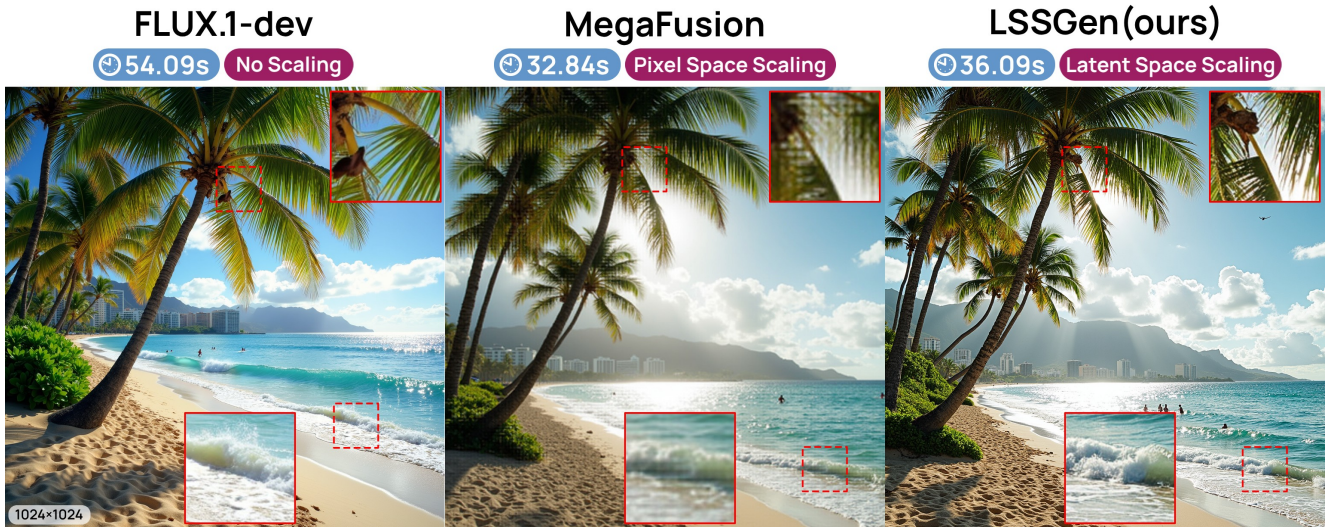


Figure 1. Comparison of image synthesis results at  $1024 \times 1024$  resolution: Our proposed method, LSSGen, leverages latent space scaling to achieve SOTA speed-quality optimality. LSSGen generates significantly higher-quality images than MegaFusion [44] and provides much faster inference than FLUX.1-dev [1].

## Abstract

Flow matching and diffusion models have shown impressive results in text-to-image generation, producing photorealistic images through an iterative denoising process. A common strategy to speed up synthesis is to perform early denoising at lower resolutions. However, traditional methods that downscale and upscale in pixel space often introduce artifacts and distortions. These issues arise when the up-scaled images are re-encoded into the latent space, leading to degraded final image quality. To address this, we propose **Latent Space Scaling Generation (LSSGen)**, a framework that performs resolution scaling directly in the latent space using a lightweight latent upsampler. Without altering the Transformer or U-Net architecture, LSSGen improves both efficiency and visual quality while supporting flexible multi-resolution generation. Our comprehensive evaluation cov-

ering text-image alignment and perceptual quality shows that LSSGen significantly outperforms conventional scaling approaches. When generating  $1024^2$  images at similar speeds, it achieves up to 246% TOPIQ score improvement. Code is available at: <https://github.com/inventec-ai-center/LSSGen>.

## 1. Introduction

Recent advances in Flow Matching (FM) and Diffusion Model (DM) have revolutionized text-to-image synthesis, achieving unprecedented image quality. Despite their differing mathematical foundations, both are implemented as continuous denoising processes based on probabilistic path learning [6]. However, this iterative nature leads to slow generation [37], and the computational cost grows quadratically with image resolution [38].

A common approach to this challenge is a progressive, coarse-to-fine generation pipeline [7, 19, 44], which generates a base-resolution image and then upscales it beyond the training resolution in pixel space. While this helps preserve semantic structure at higher resolutions, pixel-space upscaling often introduces artifacts like blurriness (Fig. 1). These artifacts arise because re-encoding the upscaled image back into the latent space distorts the underlying latent features. Without a clear understanding of these distortions, it remains difficult to resolve the speed-quality trade-off in FM and DM models.

Both DM and FM generate images by gradually transforming Gaussian noise into a clear image. This denoising process can be viewed as predicting a velocity field that moves the sample from a blurry state toward a sharp one. From an information-theoretic perspective, this progression aligns with a process that first captures low-frequency components and then incrementally adds high-frequency details [11, 33, 45]. We hypothesize that this natural blurry-to-sharp progression can be efficiently approximated by a *resolution-based* approach: starting with a low-resolution image that captures the scene’s structure, and progressively increasing the resolution to introduce fine details and improve visual clarity.

Building on this principle, we propose the **Latent Space Scaling Generation (LSSGen)** framework, which replaces the early stages of image generation with lower-resolution processing. Rather than scaling images in pixel space, LSSGen progressively upsamples features directly in the latent space using a dedicated latent upsampler. This approach avoids the artifacts commonly introduced by pixel-space resolution changes. To further address the shift in noise characteristics during scaling, we propose a novel *noise compensation* and *rescheduling* strategy. This ensures consistency between noise and data across stages and dynamically adjusts the denoising process for improved stability and image quality.

Notably, our latent upsampler is VAE-dependent, making it reusable across different flow and diffusion models that share the same VAE. This architectural decoupling allows the upsampler to work purely on VAE-derived latent features, making the upsampler agnostic to the underlying generative architecture (e.g., U-Net or Transformer). It provides a versatile, *train-once, use-across-models* solution.

Extensive experiments show that LSSGen achieves a  $1.5\times$  speedup for  $1024^2$  image generation while maintaining comparable quality across four standard evaluation metrics: GenEval [12], CLIP-IQA [43], TOPIQ [2], and NIQE [28]. At  $2048^2$  resolution, the benefits are even greater due to the quadratic increase in computational cost with image size. These results establish LSSGen as a practical and scalable solution for efficient high-resolution image synthesis. Additionally, our experiments show that LSS-

Gen offers superior global semantic preservation compared to previous methods, particularly when generating images beyond the original training resolution.

Our main contributions are summarized as follows:

- We introduce LSSGen, a latent space scaling framework applicable to a wide range of generative models. It includes a novel latent upsampling method and a noise compensation and rescheduling strategy.
- We demonstrate through extensive experiments that LSSGen achieves a strong balance between computational efficiency and image quality, outperforming baseline methods in both areas.
- We present a detailed analysis comparing pixel-space and latent-space transformations, offering fresh insights into the dynamics of multi-resolution image generation.

## 2. Related Work

Acceleration methods for DM and FM generally fall into two main categories: architecture optimization and model distillation.

**Architecture optimization:** Dynamic transformer [48] addresses computation redundancy across timesteps by introducing a Timestep-wise Dynamic Width strategy, which reduces computation by adaptively adjusting the model’s width during generation. SANA [46] employs a deep compression autoencoder to map images into a smaller feature domain, and replaces standard attention in Diffusion Transformer (DiT) [31] with linear attention for efficiency. CLEAR [25] introduce a convolution-like mechanism to reduce the complexity of the attention module. These approaches improve inference efficiency by optimizing the attention mechanisms in Transformer architectures [41]. However, they typically require training from scratch or additional fine-tuning to accommodate architectural changes.

**Model Distillation:** An alternative approach to acceleration is reducing the number of inference timesteps. Salimans and Ho [37] propose progressive distillation by merging timesteps, while Luo et al. [27] introduce consistency models that directly map noisy inputs to clean samples. Although these methods achieve significant speedups, they often compromise high-resolution fidelity by limiting the gradual error correction offered by iterative denoising. Additionally, distillation methods typically require extensive computational resources and complex training procedures. In contrast, our framework employs progressive resolution upscaling to reduce computational cost while maintaining image quality. It remains compatible with existing distillation techniques, offering complementary acceleration and quality gains when combined.

**Latent Space Modeling:** Direct sampling in pixel space poses significant computational challenges, especially for

high-resolution image generation. To address this, Rombach et al. [36] propose Latent Diffusion Model (LDM), which perform denoising in the compressed latent space of a pre-trained autoencoder [9, 21, 35]. This approach significantly reduces computational costs by leveraging dimensionality reduction. Latent-space techniques have also been adopted in flow matching [5], highlighting the general applicability of this paradigm. Recent work by Esser et al. [10] further demonstrates the scalability of Rectified Flow (RF) by integrating it into the LDM framework. However, a key challenge remains: the mismatch between pixel-space and latent-space representations complicates operations such as interpolation and scaling [13, 29], making latent manipulation less intuitive and harder to control.

**Progressive Upscaling:** Cascaded Diffusion [18] adopts a multi-resolution generation strategy to reduce the cost of high-resolution synthesis but relies on training resolution-specific super-resolution models. Self-Cascade Diffusion [14] integrates ResNet-based plugins into UNet architectures, conditioned on low-resolution latent features. While it improves generation quality, it does not reduce computational cost and requires training resolution-specific components, adding memory overhead and limiting generalizability across different architectures. MegaFusion [44] and DiffuseHigh [19] introduce progressive upsampling pipelines via pixel-space scaling and refinement. FM-Boost [38] accelerates high-resolution generation by upscaling small diffusion outputs in pixel space before applying a flow model to refine blurry regions. I-Max [8] modifies both the attention mechanism and inference pipeline of FLUX.1-dev [1] to support 4K image synthesis. However, these methods share two key limitations: (1) The encode-decode cycle often leads to detail degradation [49], and (2) Resolution scaling in pixel space introduces systematic bias into the latent space, resulting in persistent blur artifacts, even when additional noise is injected.

Our method progressively upsamples features in the latent space and uses the original diffusion or flow model to refine the final image. The upsampler is a VAE-dependent ResNet that can be reused across different generative models. This design reduces computational cost and allows for easy integration with various diffusion and flow-based architectures.

### 3. Preliminary

**Diffusion Models (DM)** generate images by iteratively denoising Gaussian noise into a target image. Let  $x_t$  denote noisy image at timestep  $t$ ,  $\epsilon \sim \mathcal{N}(0, 1)$  denote Gaussian noise,  $\beta_t \in (0, 1)$  denote a predefined noise schedule controlling noise variance at time  $t$ , and  $\bar{\alpha}$  denote the cumulative signal retention. The forward diffusion process adds

noise to the input image  $x_0$  over  $t$  timesteps, defined as:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \text{ where } \bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i). \quad (1)$$

This closed-form allows direct sampling at timestep  $t$ , improving efficiency without simulating the full Markov chain.

During training, the forward process  $q(x_{1:T}|x_0)$  corrupts the clean image  $x_0$  using Gaussian noise:  $q(x_t|x_0) := \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I})$ . The reverse process, parameterized by a neural network with model parameters  $\theta$ , learns to reconstruct the image through denoising:  $p_\theta(x_{t-1}|x_t) := \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$ , where  $\mu_\theta$  and  $\Sigma_\theta$  are the predicted mean and variance. The model is trained to minimize the difference between the predicted noise  $\epsilon_\theta$  and the true noise  $\epsilon$  via the pixel-space loss:

$$\mathcal{L}_{\text{pixel}} := \mathbb{E}_{x, \epsilon \sim \mathcal{N}(0, 1), t} [\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2]. \quad (2)$$

This formulation, introduced by Ho et al. [17] and extended by Song et al. [39], enables high-quality image generation through iterative refinement, with denoising accuracy improving at each step.

**Flow Matching (FM)** [3] formulates generation as a linear interpolation between data  $x$  and Gaussian noise  $\epsilon$ :

$$x_t := (1 - t)x + t\epsilon, \quad (3)$$

where  $t \in [0, 1]$  represents the interpolation time.

**Rectified Flow (RF)** [24, 26] further specifies the flow trajectory using a simple velocity field  $v_\theta(x_t, t) = \epsilon - x_0$ . This is modeled by a neural network parameterized by  $\theta$ , and trained using the *velocity matching objective*:

$$\mathcal{L}_{\text{flow}} := \mathbb{E}_{x, \epsilon \sim \mathcal{N}(0, 1), t} [\|v_\theta(x_t, t) - (\epsilon - x_0)\|_2^2]. \quad (4)$$

At inference, image samples are generated by solving the learned *flow ODE*, which constructs images progressively along the linear path defined by  $x_t$ . This approach offers efficient image synthesis by learning a direct transport path from noise to data, avoiding the complexity of iterative score-matching. However, accurately modeling this path remains challenging, especially when targeting complex, high-resolution distributions that require fine-grained detail and structure.

### 4. Methodology

To describe our proposed LSSGen framework, we adopt the unified formulation of diffusion and flow models introduced by Patel et al. [30]. In this setting, the generative process is defined by a parameterized ground-truth Markov generator  $F_t(x)$ , with the training objective:

$$\mathcal{L}_\theta = \mathbb{E}_{t, x} [D_{X_t}(F_t(x), F_t^\theta(x))], \quad (5)$$

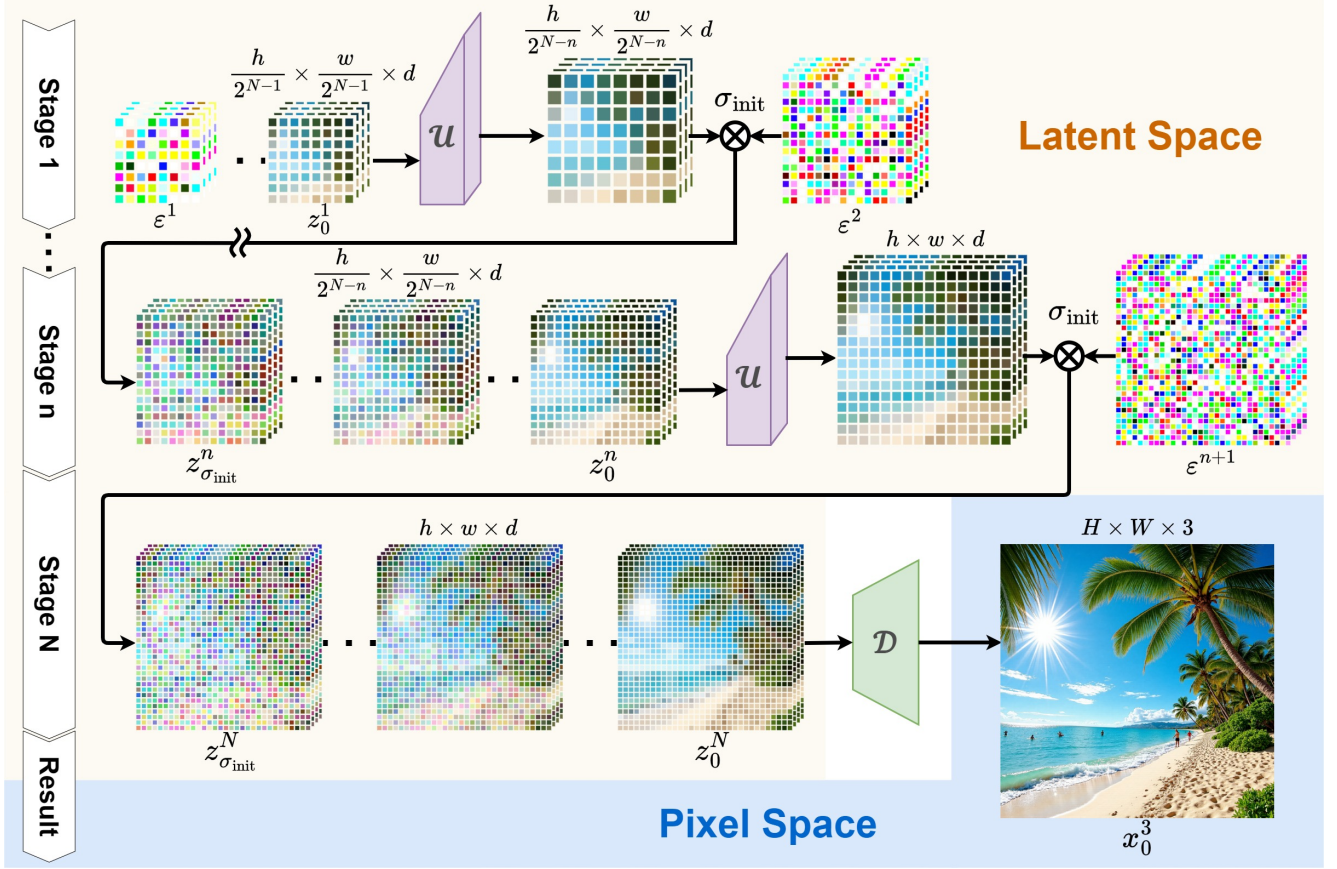


Figure 2. Overview of our Latent Space Scaling Generation (LSSGen) framework. The inference process is divided into multiple stages, implementing a progressive upscaling methodology. This approach utilizes latent space upsampling with a pretrained upsampler  $\mathcal{U}$  and incorporates a noise compensation strategy with noise timestep initialization  $\sigma_{init}$  at each stage ( $1 \rightarrow N$ ). Variables  $h$  and  $w$  correspond to latent image dimensions,  $d$  represents latent channel count, while  $H$  and  $W$  denote pixel space image dimensions.

where  $F_t^\theta(x)$  is the neural network with learning parameter  $\theta$ , and  $D$  denotes the Bregman Divergence. The loss functions of both diffusion and flow models, described in Eqs. (2) and (4), can be expressed under this unified objective. Similarly, the sampling processes in Eq. (1) and (3) can be unified by setting  $\sqrt{\bar{\alpha}_t} = 1 - t$  [40]. In practice, most diffusion and flow models encode pixel space data  $x$  into a latent representation  $z$  during training and perform inference in the latent space. Thus, we re-express the generative trajectory in terms of  $z$  as:

$$z_t = (1 - \sigma_t)z_0 + \sigma_t\epsilon, \quad (6)$$

where  $\sigma_t$  controls the noise intensity over time.

Our proposed LSSGen framework builds on this foundation by introducing three key components: (1) a probabilistic perspective for resolution-autoregressive scaling (§4.1), (2) a latent space upsampler for resolution transitions (§4.2), and (3) a timestep rescheduling mechanism to accelerate generation (§4.3).

#### 4.1. Resolution-Autoregressive Scaling

From an information-theoretic perspective, the generative path from blurry to sharp images mirrors a process that reconstructs low-frequency components first, followed by high-frequency details [11, 33, 45]. Our goal in high-resolution image synthesis follows this principle: preserve low-frequency structure and progressively add high-frequency content to enhance detail and realism.

In Eq. (6), the noise coefficient  $\sigma_t$  controls the trade-off between signal and noise ratio (SNR) [20]:

$$SNR(t) = \frac{(1 - \sigma_t)^2}{\sigma_t^2}, \quad (7)$$

which decreases over time as noise increases. Under the variance-preserving formulation [15],  $\sigma_t$  is recovered as:

$$\sigma_t = \frac{1}{1 + \sqrt{SNR(t)}}. \quad (8)$$

While diffusion and flow models usually define noise schedules using  $\bar{\alpha}_t$ , we reinterpret upsampling as a transformation of  $SNR$ . For example, scaling the resolution by

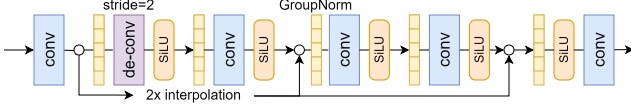


Figure 3. Upsampler architecture.

a factor of  $s = 2$  (doubling both width and height) increases the pixel count by  $4\times$ , which we approximate as reducing  $SNR(t)$  to  $\frac{1}{4}SNR(t)$ . This aligns with the view of bilinear interpolation as a smoothing process that effectively adds noise. To reflect this in the generative path, we scale the noise coefficient after upsampling as  $\sigma'_t = \frac{3}{4}\sigma_t$ , providing a more suitable initialization for denoising and better aligning with the underlying probabilistic trajectory.

The resolution scaling process is divided into  $N$  stages to better preserve image quality during upsampling. At each stage  $n \in \{1, \dots, N\}$ , the resolution is initialized as  $\frac{h}{2^{(N-1)}} \times \frac{w}{2^{(N-1)}}$  (see Fig. 2). At this scale, the signal-to-noise ratio is approximately  $\frac{1}{4}SNR$ , corresponding to a noise coefficient of  $\sigma_{\text{init}} \approx 0.75$ , assuming a linear relationship between  $SNR$  and  $\sigma$ . We adapt the probabilistic trajectory in Eq. (6) at each stage as:

$$z_{\sigma_{\text{init}}}^n \approx (1 - \sigma_{\text{init}})\mathcal{U}(z_0^{n-1}) + \sigma_{\text{init}}\epsilon^n, \quad (9)$$

where  $\mathcal{U}(z)$  is the latent upsampler from §4.2, and  $\epsilon^n \sim \mathcal{N}(0, 1)$  is Gaussian noise added at stage  $n$ .

This process iterates across  $N$  stages, with each step refining the previous output. The final high-resolution latent  $z_0^N$  is obtained by denoising the upsampled signal  $\mathcal{U}(z_0^{N-1})$ . Our progressive scaling approach avoids the cost of direct high-resolution generation at every stage. Since self-attention scales quadratically with image size, *i.e.*,  $O((H \times W)^2)$ , starting generation at low resolution and gradually scaling up provides substantial computational savings. These savings are proportional to the square of the scaling factor. A detailed analysis is provided in the supplementary material.

## 4.2. Latent Space Upsampler

We use a lightweight ResNet Upsampler  $\mathcal{U}$  (Fig. 3), with roughly 500K parameters, trained for 3 epochs on the COCO dataset[23] using a frozen, pre-trained VAE. The design supports broad compatibility with both diffusion and flow models, based on two key observations: (1) Many latent diffusion models share the same VAE, and (2) latent transformations should ideally be independent of the specific generative backbone (U-Net or DiT). This architectural decoupling allows the upsampler to operate directly on VAE-derived latent features, making it easily reusable. Since our method consistently applies  $2\times$  resolution scaling, we use ResNet [16] blocks with stride-based deconvolution [47], for efficient and effective upsampling. This design ensures the upsampler can be applied across a wide range of VAE-based generative models.

## 4.3. Timestep Schedule Shifting

To improve computational efficiency, we adopt a timestep shifting strategy inspired by Stable Diffusion 3 [10], which emphasizes the importance of early RF steps in generating high-resolution images. In our multi-stage pipeline, more denoising steps are allocated to the early, low-resolution stages where computation is cheaper. Given two resolutions, with the higher resolution  $m$  at timestep  $t_m$  and a lower resolution  $n$  at timestep  $t_n$ , we apply the following shift to align the schedules:

$$t_m = \frac{\sqrt{\frac{m}{n}}t_n}{1 + (\sqrt{\frac{m}{n}} - 1)t_n}. \quad (10)$$

This mapping increases the number of steps when spatial resolution is low, leading to a significant reduction in overall computation. The shift factor  $\sqrt{\frac{m}{n}}$  also updates the signal-to-noise ratio, allowing the noise coefficient  $\sigma$  to remain consistent with the change in resolution.

## 5. Experiments

We implement our framework using the Diffuser library [42] with PyTorch as the backend. All experiments are run on a single NVIDIA A40 GPU.

### 5.1. Experimental Setup

We evaluate LSSGen on two mainstream generative methods: Rectified Flow (RF) and Diffusion Model (DM). For RF, we use FLUX.1-dev and FLUX.1-schnell [1], two 12B-parameter models with multi-resolution capabilities. FLUX.1-dev balances image quality and efficiency, while FLUX.1-schnell is optimized for speed. We also include Stable Diffusion 3.5-medium (SD3.5-m), a 2.5B parameter model serving as a lightweight RF baseline. In the DM category, we evaluate: (1) SDXL [32], trained on resolutions from 512 to 2048; (2) Playground-v2.5 [22], an aesthetic-optimized variant of SDXL; (3) Stable Diffusion 1.5 (SD1.5) [36], designed for 512<sup>2</sup> generation; and (4) LCM-SDXL [27], a distilled model for fast inference.

We compare LSSGen with state-of-the-art (SOTA) pixel-space upsampling methods for efficient high-resolution generation. These include MegaFusion [44] and Diffuse-High [19], which directly upscale images in pixel space, as well as Self-Cascade [14], which incorporates conditional low-resolution guidance via model finetuning. All baseline results are reproduced using the officially released code.

**Evaluation metrics:** We evaluate LSSGen using four widely adopted metrics that cover both generation quality and computational efficiency. For text-to-image alignment, we use GenEval [12], which contains 553 prompts. We generate four images per prompt, following standard practice to ensure statistical robustness. To assess image qual-

Table 1. Quality and efficiency comparison on 1024<sup>2</sup> and 2048<sup>2</sup> resolutions with various generative architectures. The gray rows represent the baseline methods without a scaling strategy.

| Model                     | Scaling | Resolution | Stage Steps | Time/img↓    | Speed↑      | TFLOPs↓      | GenEval↑     | CLIP-IQA↑    | TOPIQ↑       | NIQE↓        |
|---------------------------|---------|------------|-------------|--------------|-------------|--------------|--------------|--------------|--------------|--------------|
| FLUX.1-dev [1]            | -       |            | 50          | 54.38        | 1x          | 2,991        | 0.673        | 0.887        | 0.674        | 5.622        |
| FLUX.1-dev-MegaFusion[44] | Pixel   | 1024       | 30, 20      | <b>33.57</b> | <b>1.6x</b> | <b>1,825</b> | 0.649        | 0.747        | 0.402        | 6.543        |
| LSS-FLUX.1-dev (ours)     | Latent  |            | 25, 25      | 35.79        | 1.5x        | 1,999        | <b>0.653</b> | <b>0.914</b> | <b>0.705</b> | <b>5.136</b> |
| SD3.5-m [10]              | -       |            | 40          | 14.73        | 1x          | 576          | 0.676        | 0.862        | 0.635        | 4.825        |
| SD3.5-m-MegaFusion[44]    | Pixel   | 1024       | 24, 16      | <b>9.28</b>  | <b>1.6x</b> | <b>364</b>   | 0.667        | 0.760        | 0.444        | 6.294        |
| LSS-SD3.5-m (ours)        | Latent  |            | 20, 21      | 9.97         | 1.5x        | 393          | <b>0.672</b> | <b>0.880</b> | <b>0.641</b> | <b>4.705</b> |
| Playgroundv2.5 [22]       | -       |            | 50          | 9.98         | 1x          | 608          | 0.564        | 0.940        | 0.661        | 3.881        |
| LSS-Playgroundv2.5 (ours) | Latent  | 1024       | 25, 37      | 9.11         | 1.1x        | <b>529</b>   | <b>0.559</b> | <b>0.942</b> | <b>0.665</b> | <b>3.964</b> |
| SD1.5 [36]                | -       |            | 50          | 9.21         | 1x          | 285          | 0.416        | 0.863        | 0.628        | 3.707        |
| SD1.5-MegaFusion++[44]    | Pixel   | 1024       | 40, 5, 5    | <b>4.09</b>  | <b>2.3x</b> | <b>139</b>   | 0.384        | 0.438        | 0.248        | 6.871        |
| LSS-SD1.5 (ours)          | Latent  |            | 50, 37      | 8.82         | 1x          | 284          | <b>0.451</b> | <b>0.871</b> | <b>0.609</b> | <b>4.667</b> |
| FLUX.1-schnell [1]        | -       |            | 4           | 4.77         | 1x          | 253          | 0.668        | 0.872        | 0.656        | 5.629        |
| LSS-FLUX.1-schnell (ours) | Latent  | 1024       | 2, 3        | <b>4.39</b>  | <b>1.1x</b> | <b>233</b>   | <b>0.695</b> | <b>0.916</b> | <b>0.703</b> | <b>5.412</b> |
| SDXL [32]                 | -       |            | 50          | 46.73        | 1x          | 2,415        | 0.468        | 0.699        | 0.532        | 4.725        |
| SDXL-MegaFusion [44]      | Pixel   |            | 40, 10      | <b>24.99</b> | <b>1.9x</b> | <b>1,317</b> | <b>0.566</b> | 0.716        | 0.446        | 5.817        |
| SDXL-Self-Cascade [14]    | Latent  | 2048       | 50, 50      | 56.20        | 0.8x        | 3,236        | 0.541        | 0.709        | 0.517        | <b>4.137</b> |
| SDXL-DiffuseHigh [19]     | Pixel   |            | 50, 15, 15  | 52.38        | 0.9x        | 2,518        | 0.550        | 0.681        | 0.513        | 5.960        |
| LSS-SDXL (ours)           | Latent  |            | 50, 37      | 44.61        | 1x          | 2,395        | 0.535        | <b>0.732</b> | <b>0.546</b> | 5.581        |
| FLUX.1-schnell [1]        | -       |            | 4           | 22.12        | 1x          | 917          | 0.489        | 0.655        | 0.506        | 6.481        |
| LSS-FLUX.1-schnell (ours) | Latent  | 2048       | 2, 3        | <b>19.46</b> | <b>1.1x</b> | <b>818</b>   | <b>0.657</b> | <b>0.782</b> | <b>0.586</b> | <b>5.042</b> |
| LCM-SDXL [27]             | -       |            | 8           | 5.39         | 1x          | 230          | 0.437        | 0.568        | 0.536        | 5.741        |
| LSS-LCM-SDXL(ours)        | Latent  | 2048       | 4, 6        | <b>5.00</b>  | <b>1.1x</b> | <b>206</b>   | <b>0.502</b> | <b>0.586</b> | <b>0.545</b> | <b>5.514</b> |



Text prompt: a photo of a suitcase above a skis

Figure 4. Visual comparison for different generation frameworks on SDXL with random GenEval prompts at resolution 2048<sup>2</sup>.

ity without losing high-resolution details, we avoid resize-dependent metrics like FID and instead adopt: (1) CLIP-IQA [43], which modifies CLIP [34] by replacing positional embeddings and using a ResNet backbone for resolution-invariant evaluation; (2) TOPIQ [2], designed for multi-resolution quality assessment, ideal for high-resolution outputs; (3) NIQE [28], a traditional statistical metric based on natural scene statistics, offering a non-learning-based perspective. All metrics are applied to 2,212 images generated from the GenEval prompts, providing a thorough evaluation across diverse scenarios.

## 5.2. Quality and Efficiency Comparison

**Image generation at 1024<sup>2</sup> resolution:** We evaluate LSSGen against SOTA methods across multiple generative architectures. All cascade methods start from 512<sup>2</sup> and up-scale by 1.5× or 2× to reach the target resolution. The upper section of Table 1 summarizes these results. In RF

models, LSSGen delivers a 1.5× speedup while maintaining superior image quality. On FLUX.1-dev, LSSGen improves image quality by 3-8% over the baseline, with only a slight 3% drop in GenEval alignment. In contrast, MegaFusion suffers significant quality loss, with a 16% drop in CLIP-IQA and a 40% decline in TOPIQ. Similar degradations are observed in SD3.5-m. In the DM category, LSSGen performs especially well when scaling beyond the model’s training resolutions. On Playground-v2.5, LSSGen reduces inference time by 10% with no loss in image quality. On SD1.5, it yields a 4% time reduction and an 8% gain in GenEval alignment. In comparison, MegaFusion offers 2.3× speedup but causes a 61% drop in image quality and 8% decline in alignment.

Finally, LSSGen further improves timestep-distilled models like FLUX.1-schnell, achieving a 9% speedup while boosting both image quality and alignment performance.

**Image generation at 2048<sup>2</sup> resolution:** We evaluate

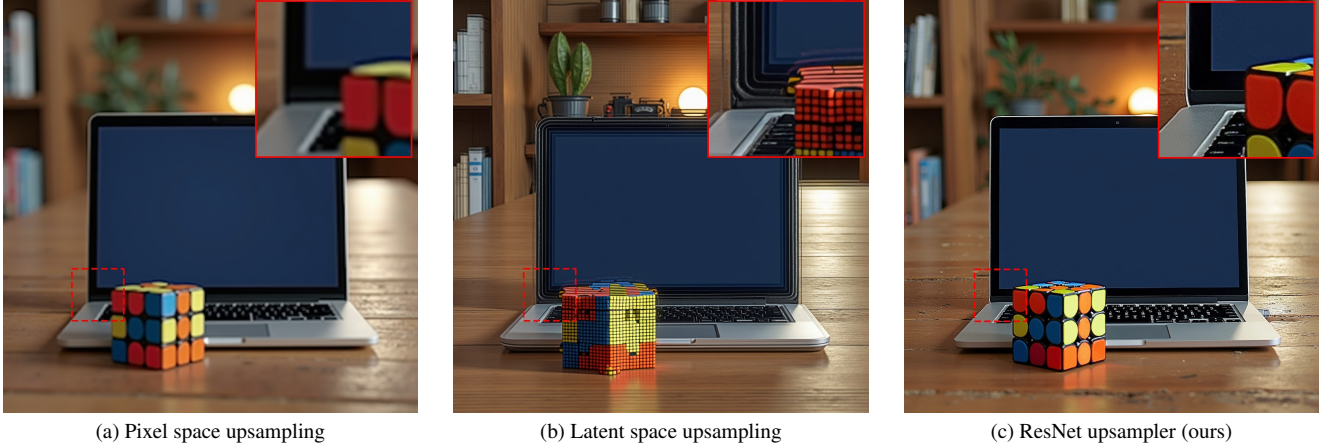


Figure 5. A visual comparison of resolution scaling methods applied within the **LSSGen** pipeline, using the FLUX.1-dev [1] backbone. The prompt is *a laptop displays a Rubik’s cube on a table*. (a) Pixel-space scaling results in significant detail degradation and blurring artifacts, even with additional denoising steps. (b) Direct latent space interpolation yields structural inconsistencies, primarily manifesting as edge duplication and boundary artifacts. (c) Our proposed ResNet upsampler preserves structural integrity with enhanced edge definition and coherent object representation, demonstrating superior perceptual quality without introducing scaling-induced artifacts.

Table 2. Comparing different scaling methods on FLUX.1-dev.

| Method               | GenEval $\uparrow$ | CLIP-IQA $\uparrow$ | TOPIQ $\uparrow$ | NIQE $\downarrow$ |
|----------------------|--------------------|---------------------|------------------|-------------------|
| Latent Interpolation | 0.605              | 0.885               | 0.651            | <b>4.588</b>      |
| Pixel Space Scaling  | 0.612              | 0.752               | 0.429            | 6.495             |
| ResNet Upsampler     | <b>0.620</b>       | <b>0.900</b>        | <b>0.670</b>     | 4.888             |

LSSGen against MegaFusion, Self-Cascade, DiffuseHigh, and standard baselines using SDXL [32], and extend our comparison to fast, timestep-distilled models like FLUX.1-schnell and LCM-SDXL, which can generate high-resolution images in just a few seconds. As shown in Table 1, MegaFusion improves text-to-image alignment by 21%, while LSSGen yields a 14% gain with SDXL. However, MegaFusion’s alignment boost comes at the cost of noticeable quality loss, with blur artifacts reflected in a lower TOPIQ score and illustrated in Fig. 4. In contrast, LSSGen maintains high perceptual quality with only a 5% increase in speed over the baseline.

LSSGen excels in high-resolution generation with flow-based models. On FLUX.1-schnell at 2048<sup>2</sup>, it improves GenEval alignment by 34%, CLIP-IQA by 19%, and TOPIQ by 16%, while reducing NIQE by 22% and speeding up inference by 1.1 $\times$ . For LCM-SDXL, a diffusion-based model, it achieves a 7% speedup, 14% better alignment, and a 3% improvement in CLIP-IQA.

### 5.3. Ablation Study

**Latent Upscaling Strategy:** We evaluate three upsampling methods: naive latent space interpolation, pixel space interpolation, and our ResNet Upsampler with FLUX.1-dev of LSSGen. As shown in Table 2, our method significantly outperforms the others in both text-to-image alignment and perceptual quality. Figure 5 visually highlights the differences: pixel-space interpolation introduces blur from re-

encoding, while naive latent interpolation leads to structural discontinuities and edge artifacts. Our ResNet Upsampler mitigates these issues with a lightweight, pretrained architecture, reinforcing the importance of well-designed latent transformations for high-fidelity multi-resolution generation, as discussed in §4.2. However, we identify a minor limitation where at higher resolutions (e.g., 2048<sup>2</sup>), the upsampled latent can become slightly over-sharpened (see supplementary). This characteristic may necessitate additional denoising steps in the final generation stage to ensure optimal quality.

**Initial Resolution:** We examine the impact of different initial resolutions in the LSSGen framework, with results shown in Table 3. Our findings show a clear trade-off between speed and text-to-image alignment. Starting at 256<sup>2</sup> yields up to 1.6 $\times$  speedup for 1024<sup>2</sup> generation but reduces alignment quality. Despite the lower starting resolution, perceptual quality remains strong across all settings. Notably, the 512<sup>2</sup> to 1024<sup>2</sup> setup retains nearly the same alignment score as the native 512<sup>2</sup> baseline, while offering a 1.5 $\times$  speedup. This suggests that text-to-image alignment is primarily influenced by the starting resolution, not the upscaling. The slight drop in alignment at very low resolutions likely reflects the hierarchical training of foundation models, which are often pretrained at lower resolutions and fine-tuned on high-quality high-resolution data [4], limiting their alignment ability.

**Initial Noise Coefficient ( $\sigma_{\text{init}}$ ):** As discussed in §4.1, LSSGen uses the initial noise coefficient  $\sigma_{\text{init}}$  to control the stochastic perturbation during upsampling. Table 4 shows how different  $\sigma_{\text{init}}$  values affect the trade-off between speed and image quality. Lower  $\sigma_{\text{init}}$  values offer greater speedups

Table 3. Impact of initial resolutions in LSSGen on FLUX.1-dev. All metrics are evaluated at the target resolution.

| Init Resolution | Target Resolution | Stage $N$ | Time/img↓    | Speed↑      | GenEval↑     | CLIP-IQA↑    | TOPIQ↑       | NIQE↓        |
|-----------------|-------------------|-----------|--------------|-------------|--------------|--------------|--------------|--------------|
| 256             | 256               | -         | 9.22         | -           | 0.584        | 0.854        | 0.747        | 8.729        |
| 256             | 1024              | 3         | <b>34.93</b> | <b>1.6x</b> | 0.594        | <b>0.932</b> | 0.694        | 5.218        |
| 512             | 512               | -         | 17.61        | -           | 0.652        | 0.899        | 0.728        | 5.820        |
| 512             | 1024              | 2         | 36.09        | 1.5x        | <b>0.653</b> | 0.914        | <b>0.705</b> | <b>5.136</b> |
| 1024            | 1024              | -         | 54.38        | 1x          | 0.673        | 0.887        | 0.674        | 5.622        |

Table 4. Impact of different initial sigma in LSSGen on SD3.5-m.

| $\sigma_{\text{init}}$ | Time/img↓   | Speed↑      | GenEval↑     | CLIP-IQA↑    | TOPIQ↑       | NIQE↓        |
|------------------------|-------------|-------------|--------------|--------------|--------------|--------------|
| 0.3                    | <b>4.59</b> | <b>3.2x</b> | 0.659        | 0.874        | 0.626        | 5.931        |
| 0.4                    | 5.30        | 2.8x        | 0.659        | 0.884        | 0.631        | 5.541        |
| 0.5                    | 6.39        | 2.3x        | 0.660        | <b>0.885</b> | 0.636        | 5.217        |
| 0.6                    | 7.46        | 2x          | 0.663        | 0.884        | 0.639        | 5.025        |
| 0.7                    | 8.90        | 1.7x        | 0.675        | 0.882        | <b>0.641</b> | 4.848        |
| 0.75                   | 9.97        | 1.5x        | 0.672        | 0.880        | <b>0.641</b> | 4.705        |
| 0.8                    | 11.02       | 1.3x        | 0.673        | 0.879        | <b>0.641</b> | 4.718        |
| 0.9                    | 13.53       | 1.1x        | <b>0.677</b> | 0.876        | 0.640        | <b>4.622</b> |
| 1.0                    | 14.73       | 1x          | 0.676        | 0.862        | 0.635        | 4.825        |

with minimal loss in semantic alignment. As  $\sigma_{\text{init}}$  approaches the theoretical optimal value of 0.75, perceptual quality improves steadily, though at the cost of reduced efficiency. We find that values in the range [0.7, 0.8] offer the best balance between image quality and a 1.3× to 1.7× speedup over the baseline. This supports our theoretical insight that  $\sigma_{\text{init}} \approx 0.75$  is optimal for half-resolution initialization. However, the framework also offers flexibility for applications where throughput is the primary concern. By setting  $\sigma_{\text{init}}$  to lower values, a slight degradation in perceptual quality can be exchanged for more significant computational gains. For example, using a  $\sigma_{\text{init}}$  between 0.3 and 0.5 can achieve a 2.3× to 3.2× speedup. Conversely,  $\sigma_{\text{init}}$  values exceeding 0.8 yield diminishing quality gains with significantly higher computational cost, confirming that our framework effectively captures the trade-offs in resolution-aware noise scheduling.

**Impact of Schedule Shifting:** We evaluate the effect of the shifting strategy from §4.3 in the LSSGen framework using SD3.5-m, with results shown in Table 5. Shifting reduces inference time by 26%, achieving a 1.5× speedup over the baseline—compared to 1.1× without shifting. It also improves perceptual quality by about 3%. Although NIQE slightly worsens by 3%, this is a reasonable trade-off for the overall gains in efficiency and quality. These results support our theoretical insight that concentrating more timesteps in early, low-resolution stages improves efficiency without compromising image fidelity, making shifting a natural fit for resolution-aware generation.

**Shorten intermediate steps:** We investigate the possibility of reducing sampling steps in lower-resolution where the denoising process is expected to be less complex. The sampling step reduction at lower resolution stages is according

Table 5. The effects of the schedule shifting (SD3.5-m backbone) and shortening early stage timesteps (FLUX.1-dev backbone).

| Setting       | Time/img↓      | Speed↑      | GenEval↑     | CLIP-IQA↑    | TOPIQ↑       | NIQE↓        |
|---------------|----------------|-------------|--------------|--------------|--------------|--------------|
| Shift         | <b>9.97 s</b>  | <b>1.5x</b> | <b>0.672</b> | <b>0.880</b> | <b>0.641</b> | 4.705        |
| No Shift      | 13.53 s        | 1.1x        | 0.653        | 0.852        | 0.621        | <b>4.546</b> |
| Shorten Steps | <b>35.79 s</b> | <b>1.5x</b> | 0.653        | <b>0.914</b> | <b>0.705</b> | <b>5.136</b> |
| Regular Steps | 44.81 s        | 1.2x        | <b>0.658</b> | 0.910        | 0.700        | 5.246        |

to the stage  $n$ , with  $T' = T / (2^{(N-n)})$ . Table 5 demonstrates that shortened steps at early stages of LSSGen yield efficiency gains (1.2× to 1.5×) with minimal impact on generation quality. Computational gains emerge with negligible quality impact, illuminating the potential to compress the generation process in a resolution-dependent manner.

## 6. Conclusion

This paper presents LSSGen, a general and efficient framework for accelerating inference in both diffusion and flow-based generative models. The core idea is to shift the early generation steps to a low-resolution latent space and progressively upscale the result to the target resolution. This design enables high-resolution synthesis with significantly reduced computation while preserving visual fidelity.

Unlike previous pixel-space upsampling methods, LSSGen operates entirely in latent space, requiring no architectural changes or model retraining. It offers a lightweight and generalizable ResNet-based latent upsampler and a principled schedule-shifting mechanism to maximize efficiency during denoising. Extensive experiments across multiple SOTA models (e.g., FLUX, SDXL, SD1.5, Playground-v2.5, and LCM) and resolutions ( $1024^2$  and  $2048^2$ ) show that LSSGen achieves up to 246% improvement in perceptual quality, 1.5× speedup, and stronger semantic alignment, even on models not trained for high resolutions.

In addition to strong empirical results, the paper provides theoretical formulations for resolution-aware SNR adjustment, initialization strategies, and probabilistic path planning for dynamic latent upscaling. These insights help bridge the gap between efficiency and fidelity.

**Future work** includes extending LSSGen to more complex tasks such as video generation, image editing, and inpainting, where latent-space resolution dynamics are even more critical. We also plan to explore adaptive scaling schedules for further gains.

## References

- [1] Black Forest Labs. FLUX.1: Text-to-image Generation Model. <https://github.com/black-forest-labs/flux>, 2024. Released: August 2, 2024. 1, 3, 5, 6, 7
- [2] Chaofeng Chen, Jiadi Mo, Jingwen Hou, Haoning Wu, Liang Liao, Wenxiu Sun, Qiong Yan, and Weisi Lin. TOPIQ: A top-down approach from semantics to distortions for image quality assessment. *IEEE Transactions on Image Processing*, 2024. 2, 6
- [3] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018. 3
- [4] Xiaoliang Dai, Ji Hou, Chih-Yao Ma, Sam Tsai, Jialiang Wang, Rui Wang, Peizhao Zhang, Simon Vandenhende, Xiao-fang Wang, Abhimanyu Dubey, et al. Emu: Enhancing image generation models using photogenic needles in a haystack. *arXiv preprint arXiv:2309.15807*, 2023. 7
- [5] Quan Dao, Hao Phung, Binh Nguyen, and Anh Tran. Flow matching in latent space. *arXiv preprint arXiv:2307.08698*, 2023. 3
- [6] Carles Domingo-Enrich, Michal Drozdal, Brian Karrer, and Ricky TQ Chen. Adjoint Matching: Fine-tuning Flow and Diffusion Generative Models with Memoryless Stochastic Optimal Control. In *Proceedings of the International Conference on Learning Representations*, 2025. 1
- [7] Ruoyi Du, Dongliang Chang, Timothy Hospedales, Yi-Zhe Song, and Zhanyu Ma. Demofusion: Democratising high-resolution image generation with no \$\$\$\$. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6159–6168, 2024. 2
- [8] Ruoyi Du, Dongyang Liu, Le Zhuo, Qin Qi, Hongsheng Li, Zhanyu Ma, and Peng Gao. I-max: Maximize the resolution potential of pre-trained rectified flow transformers with projected flow. *arXiv preprint arXiv:2410.07536*, 2024. 3
- [9] Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12873–12883, 2021. 3
- [10] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Proceedings of the International Conference on Machine Learning*, 2024. 3, 5, 6
- [11] Fabian Falck, Teodora Pandeava, Kiarash Zahirmia, Rachel Lawrence, Richard Turner, Edward Meeds, Javier Zazo, and Sushrut Karmalkar. A Fourier Space Perspective on Diffusion Models. *arXiv preprint arXiv:2505.11278*, 2025. 2, 4
- [12] Dhruva Ghosh, Hannaneh Hajishirzi, and Ludwig Schmidt. Geneval: An object-focused framework for evaluating text-to-image alignment. *Advances in Neural Information Processing Systems*, 36:52132–52152, 2023. 2, 5
- [13] Jiayi Guo, Xingqian Xu, Yifan Pu, Zanlin Ni, Chaofei Wang, Manushree Vasu, Shiji Song, Gao Huang, and Humphrey Shi. Smooth diffusion: Crafting smooth latent spaces in diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7548–7558, 2024. 3
- [14] Lanqing Guo, Yingqing He, Haoxin Chen, Menghan Xia, Xiaodong Cun, Yufei Wang, Siyu Huang, Yong Zhang, Xintao Wang, Qifeng Chen, et al. Make a cheap scaling: A self-cascade diffusion model for higher-resolution adaptation. In *Proceedings of the European Conference on Computer Vision*, pages 39–55, 2024. 3, 5, 6
- [15] Tiankai Hang, Shuyang Gu, Xin Geng, and Baining Guo. Improved noise schedule for diffusion training. *arXiv preprint arXiv:2407.03297*, 2024. 4
- [16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 5
- [17] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020. 3
- [18] Jonathan Ho, Chitwan Saharia, William Chan, David J Fleet, Mohammad Norouzi, and Tim Salimans. Cascaded diffusion models for high fidelity image generation. *Journal of Machine Learning Research*, 23(47):1–33, 2022. 3
- [19] Younghyun Kim, Geunmin Hwang, Junyu Zhang, and Eunbyung Park. Diffusehigh: Training-free progressive high-resolution image synthesis through structure guidance. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4338–4346, 2025. 2, 3, 5, 6
- [20] Diederik Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. *Advances in Neural Information Processing Systems*, 34:21696–21707, 2021. 4
- [21] Diederik P Kingma and Max Welling. Auto-encoding variational Bayes. *Proceedings of the International Conference on Learning Representations*, 2014. 3
- [22] Daiqing Li, Aleks Kamko, Ehsan Akhgari, Ali Sabet, Linmiao Xu, and Suhail Doshi. Playground v2.5: Three insights towards enhancing aesthetic quality in text-to-image generation. *arXiv preprint arXiv:2402.17245*, 2024. 5, 6
- [23] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Proceedings of the European Conference on Computer Vision*, pages 740–755, 2014. 5
- [24] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow Matching for Generative Modeling. In *Proceedings of the International Conference on Learning Representations*, 2023. 3
- [25] Songhua Liu, Zhenxiong Tan, and Xinchao Wang. CLEAR: Conv-Like Linearization Revisits Pre-Trained Diffusion Transformers Up. *arXiv preprint arXiv:2412.16112*, 2024. 2
- [26] Xingchao Liu, Chengyue Gong, et al. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. In *Proceedings of the International Conference on Learning Representations*, 2023. 3
- [27] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-

- resolution images with few-step inference. *arXiv preprint arXiv:2310.04378*, 2023. 2, 5, 6
- [28] Anish Mittal, Rajiv Soundararajan, and Alan C Bovik. Making a “completely blind” image quality analyzer. *IEEE Signal Processing Letters*, 20(3):209–212, 2012. 2, 6
  - [29] Yong-Hyun Park, Mingi Kwon, Jaewoong Choi, Junghyo Jo, and Youngjung Uh. Understanding the latent space of diffusion models through the lens of Riemannian geometry. *Advances in Neural Information Processing Systems*, 36: 24129–24142, 2023. 3
  - [30] Zeeshan Patel, James DeLoye, and Lance Mathias. Exploring Diffusion and Flow Matching Under Generator Matching. *arXiv preprint arXiv:2412.11024*, 2024. 3
  - [31] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023. 2
  - [32] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis. In *Proceedings of the International Conference on Learning Representations*, 2024. 5, 6, 7
  - [33] Yurui Qian, Qi Cai, Yingwei Pan, Yehao Li, Ting Yao, Qibin Sun, and Tao Mei. Boosting diffusion models with moving average sampling in frequency domain. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8911–8920, 2024. 2, 4
  - [34] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *Proceedings of the International Conference on Machine Learning*, pages 8748–8763, 2021. 6
  - [35] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the International Conference on Machine Learning*, pages 1278–1286, 2014. 3
  - [36] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022. 3, 5, 6
  - [37] Tim Salimans and Jonathan Ho. Progressive Distillation for Fast Sampling of Diffusion Models. In *Proceedings of the International Conference on Learning Representations*, 2022. 1, 2
  - [38] Johannes Schusterbauer, Ming Gui, Pingchuan Ma, Nick Stracke, Stefan Andreas Baumann, Vincent Tao Hu, and Björn Ommer. FMBoost: Boosting Latent Diffusion with Flow Matching. In *Proceedings of the European Conference on Computer Vision*, pages 338–355, 2024. 1, 3
  - [39] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models. In *Proceedings of the International Conference on Learning Representations*, 2021. 3
  - [40] Kaiyu Song and Hanjiang Lai. Unraveling the Connections between Flow Matching and Diffusion Probabilistic Models in Training-free Conditional Generation. *arXiv preprint arXiv:2411.07625*, 2024. 4
  - [41] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017. 2
  - [42] Patrick von Platen, Suraj Patil, Anton Lozhkov, Pedro Cuenca, Nathan Lambert, Kashif Rasul, Mishig Davaadorj, Dhruv Nair, Sayak Paul, William Berman, Yiyi Xu, Steven Liu, and Thomas Wolf. Diffusers: State-of-the-art diffusion models. <https://github.com/huggingface/diffusers>, 2022. 5
  - [43] Jianyi Wang, Kelvin CK Chan, and Chen Change Loy. Exploring clip for assessing the look and feel of images. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2555–2563, 2023. 2, 6
  - [44] Haoning Wu, Shaocheng Shen, Qiang Hu, Xiaoyun Zhang, Ya Zhang, and Yanfeng Wang. MegaFusion: Extend Diffusion Models towards Higher-resolution Image Generation without Further Tuning. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2025. 1, 2, 3, 5, 6
  - [45] Wei Wu, Qingnan Fan, Shuai Qin, Hong Gu, Ruoyu Zhao, and Antoni B Chan. FreeDiff: Progressive Frequency Truncation for Image Editing with Diffusion Models. In *Proceedings of the European Conference on Computer Vision*, pages 194–209, 2024. 2, 4
  - [46] Enze Xie, Junsong Chen, Junyu Chen, Han Cai, Haotian Tang, Yujun Lin, Zhekai Zhang, Muyang Li, Ligeng Zhu, Yao Lu, et al. Sana: Efficient high-resolution image synthesis with linear diffusion transformers. In *Proceedings of the International Conference on Learning Representations*, 2025. 2
  - [47] Matthew D Zeiler, Dilip Krishnan, Graham W Taylor, and Rob Fergus. Deconvolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2528–2535, 2010. 5
  - [48] Wangbo Zhao, Yizeng Han, Jiasheng Tang, Kai Wang, Yibing Song, Gao Huang, Fan Wang, and Yang You. Dynamic diffusion transformer. In *Proceedings of the International Conference on Learning Representations*, 2025. 2
  - [49] Zixin Zhu, Xuelu Feng, Dongdong Chen, Jianmin Bao, Le Wang, Yinpeng Chen, Lu Yuan, and Gang Hua. Designing a better asymmetric vqgan for stablediffusion. *arXiv preprint arXiv:2306.04632*, 2023. 3